

EEP-2 Software Exploration Program

Week-wise Laboratory Activities & Applied Programming Tasks

Platform: Linux + Python + GitHub

Introduction

The course follows a practical and tool-driven learning approach where each activity directly contributes toward the final capstone project. Every week introduces a new computational tool or software engineering concept that later becomes part of a larger integrated retrieval system.

The overall progression of the course gradually leads students toward building an Intelligent Information Retrieval System, where students combine concepts such as:

- text processing
- indexing
- ranking
- query handling
- fuzzy matching (exploration)

into a complete software system.

Each performing week (Weeks 1 through 9) includes a graded practical activity carrying 5 marks, giving a total of 45 marks across the course. These marks form part of the Task Based Assessment component of the grading scheme.

Two-Track Structure

Because this course is designed for 2nd semester students, every week is split into two tracks so that students are not overwhelmed by content that exceeds the scope of their current programming maturity.

- **Core Track** — Mandatory. Contains the essential concepts and the 5-mark weekly mini project. Designed to be completable within the weekly lab time.
- **Exploration Track** — Optional and ungraded. Contains advanced or out-of-scope content for curious students. Performing well here does not affect marks; not attempting it does not lose marks.

Week 1 — Linux Environment & Filesystem Navigation

[Core] Terminal Exploration & Shell Basics

[Core] Filesystem Navigation Drill

[Core] File & Directory Operations Practice

[Core] Manual Pages & Command Documentation

[Exploration] Custom Shell Prompt with PS1

[Exploration] Symbolic Links, Permissions & PATH

Mini Project — **Workspace Setup (5 Marks)**

Week 2 — Command-Line Tools & Text Processing

[Core] File Searching with find

[Core] Text Filtering with grep, sort, uniq, wc

[Core] Pipelines & I/O Redirection

[Core] Server Log Exploration Drill

[Exploration] Regular Expressions with grep -E

[Exploration] Text Transformation with sed and awk

[Exploration] cut, paste, tr — Column Surgery

Mini Project — **Command-Line Log Analyzer (5 Marks)**

Week 3 — Shell Scripting & Automation

[Core] Variables, Arguments & User Input

[Core] Conditional Execution (if/elif/else)

[Core] Loop-Based Processing (for, while)

[Core] Simple Batch Operations

[Exploration] case Statements & Pattern Matching

[Exploration] Logging, Cron Jobs & Script Hardening

Mini Project — **Automated File Organizer (5 Marks)**

Week 4 — Version Control with Git

[Core] Repository Initialization & Staging

[Core] Commit Workflow & Change Tracking

[Core] Project History Exploration (log, diff, show)

[Core] Branching & Merging Basics

[Exploration] Interactive Staging, Amend, Reset, Revert

[Exploration] Tagging, Conflicts, blame & bisect

Mini Project — **Local Repository Recovery Challenge (5 Marks)**

Week 5 — Collaborative Development with GitHub

[Core] Remote Repository Setup
[Core] Push / Pull / Fetch Workflow
[Core] Simple Merge Conflict Resolution
[Core] Pull Request Basics
[Exploration] SSH Keys, Review Etiquette, Branch Protection
[Exploration] Rebasing vs Merging, Long-Term Forks
Mini Project — Team Repository Recovery Challenge (5 Marks)

Week 6 — Python Foundations & Computational Thinking

[Core] Python Environment & Virtual Environments
[Core] Variables, Operators & Data Types
[Core] Conditions & Loops
[Core] File & Text Processing in Python
[Exploration] List Comprehensions & collections.Counter
[Exploration] argparse, Type Hints & Docstrings
Mini Project — Interactive Text Corpus Analyzer (5 Marks)

Week 7 — Web Scraping & Dataset Collection

[Core] HTTP Requests with requests
[Core] Extracting Plain Text from HTML with BeautifulSoup
[Core] Saving Articles as JSON
[Core] Scraping a List of URLs
[Exploration] Structured Article Parsing (sections, summary, links)
[Exploration] Following Internal Links / Crawling
[Exploration] Retry Logic, logging module, Wikipedia API, robots.txt
Mini Project — Wikipedia Dataset Collector (5 Marks)

Week 8 — Functions, Modularity & Data Structures

[Core] Function Design, Parameters & Return Values
[Core] Lists, Dictionaries, Sets & Tuples for Text Data
[Core] Multi-File Program Organization (modules & imports)
[Core] Structured Data Processing Pipeline
[Exploration] Unit Testing with pytest
[Exploration] Dataclasses, Generators, CLI with argparse
Mini Project — Modular Document Metadata Organizer (5 Marks)

Week 9 — Linear vs Inverted Index & Frequency Counting

[Core] Linear Search Across Documents

[Core] Building an Inverted Index

[Core] Comparing Linear vs Inverted Index

[Core] Frequency Counting per Document

[Exploration] Stopword Removal

[Exploration] TF-IDF Ranking

[Exploration] Positional Index, Boolean Queries, Phrase Queries

[Exploration] Fuzzy Matching (Edit Distance, difflib)

[Exploration] Binary Search on Sorted Vocabulary

Mini Project — Inverted Index Builder & Lookup Tool (5 Marks: 3 + 2)

Week 10 — Indexing & Search Architecture (Capstone Transition)

From the start of this week students transition toward the capstone theme — full freedom on the index implementation.

[Core] Text Cleaning & Basic Tokenization

[Core] Posting Lists & Document Mapping

[Core] Query Retrieval Workflow Integration

[Core] Capstone Planning & Architecture Sketch

[Exploration] Pick stretch features from Week 9 Exploration to add to capstone

Mini Project — Ranked Query Engine (capstone foundation, ungraded this week)

Weeks 11 & 12 — Final Capstone Development & Demonstration

Final Capstone — Intelligent Wikipedia Search Engine

- Dataset cleaning & normalization
- Inverted index + ranking
- CLI interface (mandatory)
- Final demonstration & code walkthrough
- **Bonus:** Query handling with autocomplete/suggestions & fuzzy implementation

Total Graded Marks: 45 (Weeks 1–9, 5 marks per week). Capstone (Weeks 10–12): Forms basis of capstone grade (2 points of overall course grade).

WEEK 1 — Linux Environment & Filesystem Navigation

Conceptual Focus *Setting up the Linux working environment, understanding the shell as an interface to the operating system, navigating the filesystem hierarchy, and performing fundamental file operations. Students learn to treat the terminal as the primary developer tool — the same surface they will use for the rest of the course.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. Terminal Exploration & Shell Basics Students open a terminal (native Ubuntu, WSL, or lab machine) and explore the shell as a command interpreter. They identify the prompt structure (user@host:path\$), check the active shell with `echo $SHELL`, view environment variables with `env`, and inspect their identity using `whoami`, `id`, `hostname`, and `uname -a`. They use history, up-arrow recall, and `clear` to understand session state.

2. Filesystem Navigation Drill Students navigate the Linux directory tree using `pwd`, `cd`, `ls` (with flags `-l`, `-a`, `-h`). They explore standard directories — `/home`, `/tmp`, `/etc` — and explain in their own words what each contains. They practise absolute vs relative paths, and use `~`, `.`, `..`, and `-` (previous directory).

3. File & Directory Operations Practice Students create, copy, move, rename, and delete files and directories using `touch`, `mkdir -p`, `cp -r`, `mv`, `rm`, `rmdir`. They view file contents with `cat`, `less`, `head`, `tail`, and inspect file metadata with `stat`, `file`, `wc`. They learn the importance of `rm -i` for safety.

4. Manual Pages & Command Documentation Students learn to be self-sufficient by using `man`, `--help`, and `which` to discover and understand commands without asking the instructor. They are given two unfamiliar commands (e.g., `tree`, `du`) and must explain each by reading its man page.

5. Mini Project — Developer Workspace Setup (5 Marks) Each student builds a clean, reproducible developer workspace on their machine. They must:

- Create a structured directory tree under `~/eep-software/` containing folders `week-01/` through `week-12/`, `notes/`, `scripts/`, and `capstone/`.
- Populate each weekly folder with a `README.md` describing the week's topic (written using `nano`).
- Add at least two simple aliases to `.bashrc` (e.g., `ll`, `..`).

- Produce a one-page text report (workspace-report.txt) listing the full directory tree (using `ls -R` or `tree`) and the contents of their `.bashrc` additions.

The workspace built this week becomes the home for every subsequent assignment.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

For students who want to dig deeper into the Linux environment beyond what the core lab covers.

1. Custom Shell Prompt with PS1 Explore the `PS1` environment variable. Build a custom prompt that shows the current git branch, time, and a colored username. Research what `\u`, `\h`, `\w`, and ANSI color codes mean and experiment with them in `.bashrc`.

2. Advanced `ls` and `find` Flags Study `ls -R`, `ls -t`, `ls --color=auto`, and `find` with `-mtime`, `-size`, `-exec`. Write a one-liner that finds all files in `/home` modified in the last 7 days and prints them sorted by size.

3. Symbolic Links, Permissions & File Types Explore `ln -s` to create symbolic links, `chmod` with octal (755, 644) and symbolic (`u+x`) notation, and `chown`. Read about the difference between regular files, directories, symbolic links, and block/character devices using `ls -l` output.

4. Environment Variables & `PATH` Understand how `PATH` works, why which can find some commands but not others, and how to add a custom directory to your `PATH` permanently via `.bashrc`.

WEEK 2 — Command-Line Tools & Text Processing

Conceptual Focus *Treating text as the universal data format on Linux. Students learn that small, sharp tools — chained together — can extract meaningful information from messy data. This is the same philosophy that underlies a search engine: input text → filter → transform → output.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. File Searching with find Students use find with the most common predicates -name, -type, and -size to locate files across their Week 1 workspace. They search for all README.md files and all files larger than 1 KB. They are introduced to locate as a faster alternative for indexed searches.

2. Text Filtering with grep, sort, uniq, wc Students are given a small text corpus (a downloaded plain-text book or a sample log file). Using grep with flags -i, -n, -v, -c, they extract lines matching patterns. They use sort to order data, uniq -c to count duplicates, and wc -l/-w/-c to measure size.

3. Pipelines & I/O Redirection Students chain commands with |, redirect output using > and >>, and read input with <. They build progressively longer pipelines — for example, 'find the ten most frequent words in this text file' — and explain each stage's contribution.

4. Server Log Exploration Drill Students are given a sample server access log. They answer specific questions using only command-line tools: How many unique IP addresses appear? What is the most-requested URL? How many 404 responses occurred? Each answer is produced by a single one-line pipeline.

5. Mini Project — Command-Line Log Analyzer (5 Marks) Students build a small toolkit of one-liner pipelines (saved in a single shell script analyze.sh or as separate .sh files) that analyses a server log file and produces a short report. Requirements:

- At least four different analyses: top IP addresses, top URLs, status code distribution, and total request count.
- Each analysis is a single pipeline using only the tools learned this week.
- Output is collected into a plain-text file report.txt using > and >>.
- Tested on a sample log file provided by the instructor.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

Advanced text manipulation tools for students comfortable with the basics.

1. Regular Expressions with grep -E and egrep Move beyond literal string matching. Learn extended regex: character classes [a-z], quantifiers (*, +, ?), anchors (^, \$), and alternation (|). Practise extracting email addresses, IP addresses, and date patterns from a sample text file.

2. Text Transformation with sed and awk Explore sed for stream substitution (s/old/new/g) and awk for column-based processing ({print \$2}). Write a one-liner that uses awk to compute the total bytes transferred from a web server access log.

3. cut, paste, tr — Column Surgery Use cut -d',' -f1,3 to extract specific CSV columns, paste to merge files line-by-line, and tr to translate or delete characters (e.g., tr -d ',' to remove commas, tr 'A-Z' 'a-z' to lowercase a file).

4. Building Reusable One-Liners Take your favorite analysis pipelines and save them as shell aliases or functions in .bashrc. Learn how shell functions differ from aliases.

WEEK 3 — Shell Scripting & Automation

Conceptual Focus *Transitioning from one-liners to reusable, parameterised programs. Students learn that the same pipelines from Week 2 become powerful when wrapped in scripts that take arguments, make decisions, and loop over data.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

- 1. Variables, Arguments & User Input** Students write their first .sh scripts using the #!/bin/bash shebang. They declare variables, use command substitution (\$(...)), read user input with read, and access command-line arguments via \$1, \$2, \$#. They make their scripts executable with chmod +x.
- 2. Conditional Execution** Students use if/elif/else and [[...]] test expressions to make scripts branch on conditions. They write scripts that check whether a file exists, whether a directory is empty, or whether a given string matches a pattern. They learn exit codes (\$?).
- 3. Loop-Based Processing** Students use for loops over file globs and while read loops over file contents. They write a script that iterates over every .txt file in a folder and prints its line count — combining loops with the Week 2 wc tool.
- 4. Simple Batch Operations** Students perform real batch tasks: bulk-renaming files (e.g., adding date prefixes) and bulk-counting lines across many files. They practise dry-runs (echo before the command) before destructive operations.
- 5. Mini Project — Automated File Organizer (5 Marks)** Students write a Bash script organize.sh that takes a directory path as an argument and automatically sorts its contents into sub-folders by file type — Documents/, Images/, Code/, Other/. Requirements:
 - Validate that the input directory exists (using conditionals from this week).
 - Move files into the correct sub-folder based on extension (.txt, .pdf → Documents/; .jpg, .png → Images/; .py, .sh → Code/; everything else → Other/).
 - Print a summary at the end showing how many files were moved into each folder.
 - Tested on a small messy folder the student creates themselves with at least 10 mixed files.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

For students who want their scripts to be more robust and production-ready.

1. case Statements & String Pattern Matching Replace long if/elif chains with case ... esac. Useful when classifying file extensions or handling many command-line options. Try rewriting your organizer using case.

2. Logging & Timestamped Output Add proper logging to your scripts. Use date '+%Y-%m-%d %H:%M:%S' to prepend timestamps to log lines, write to a log file under ~/eep-software/scripts/logs/, and learn the convention of writing errors to stderr with >&2.

3. Filename Collision Handling What happens when two files with the same name end up in the same destination folder? Implement a numeric-suffix renaming scheme (file.txt, file_1.txt, file_2.txt) using a while loop.

4. Cron Jobs for Scheduled Automation Learn about cron and crontab -e. Schedule your organizer script to run daily on your Downloads folder. Understand the five-field crontab syntax (minute, hour, day, month, weekday).

5. Script Hardening — set -euo pipefail Investigate why professional Bash scripts often start with set -euo pipefail. Understand strict error handling and how it makes scripts safer.

WEEK 4 — Version Control with Git

Conceptual Focus *Treating code and writing as a tracked, recoverable history rather than a sequence of overwrites. Students learn to think in commits, branches, and diffs — the same mental model professional developers use every day.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. Repository Initialization & Staging Students initialise their existing ~/eep-software/ workspace as a Git repository with git init. They configure identity with git config --global, write a simple .gitignore (excluding logs and temp files), and observe the three-area model — working directory, staging area, repository — using git status.

2. Commit Workflow & Change Tracking Students stage and commit their Week 1–3 work in logically grouped commits with meaningful messages. They practise git add, git commit -m, and write commit messages with a short summary line.

3. Project History Exploration Students explore their own history using git log, git log --oneline, git show <hash>, and git diff. They reconstruct the story of how their Week 3 organizer script evolved.

4. Branching & Merging Basics Students create feature branches (git branch, git switch -c), make changes, and merge back to main. They understand what fast-forward and three-way merges look like by reading git log --oneline --graph.

5. Mini Project — Local Repository Recovery Challenge (5 Marks) The instructor provides each student with a 'broken' repository archive containing:

- Files committed that should have been ignored (large binaries, log files).
- A commit with a typo in the message that needs amending.
- Two branches that need to be merged cleanly.
- Students must hand back a cleaned repository with: a sensible .gitignore, a clean git log, and a RECOVERY.md file documenting every Git command they ran and why.

Marks are awarded for both the final state and the explanation quality.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

Advanced Git topics for students who want to use Git like a professional.

- 1. Interactive Staging with git add -p** Learn to stage only parts of a file using `git add --patch`. This is invaluable when you've made multiple unrelated changes and want clean, focused commits.
- 2. Rewriting History — amend, reset, revert** Explore `git commit --amend`, `git reset --soft/--mixed/--hard`, and `git revert`. Understand the difference between rewriting history and adding a reversal commit.
- 3. Tagging Releases** Learn `git tag v1.0` to mark stable points in history, and `git checkout <tag>` to inspect them. Tags are how real projects mark released versions.
- 4. Resolving Merge Conflicts** Deliberately create a merge conflict locally by editing the same lines on two different branches. Practise resolving it manually, marking with `git add`, and completing the merge.
- 5. git blame & git bisect** `git blame` tells you who last changed each line. `git bisect` helps you find which commit introduced a bug using binary search. Both are powerful debugging tools.

WEEK 5 — Collaborative Development with GitHub

Conceptual Focus *Extending Git from a solo tool to a collaboration platform. Students learn the workflows real software teams use — remotes, pull requests, and resolving conflicts between humans, not just between branches.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. Remote Repository Setup Students create a GitHub account (if they don't have one), set up authentication (HTTPS with a personal access token is acceptable; SSH keys are encouraged but optional), and push their Week 4 workspace repository to a new GitHub repository. They verify it appears online.

2. Push / Pull / Fetch Workflow Students simulate working from two machines — they clone their own repo into a second directory, make commits in one, push, then pull from the other. They observe what git fetch does versus git pull, and use git remote -v to inspect their remotes.

3. Simple Merge Conflict Resolution Students pair up. Both clone a shared instructor-provided repository, both edit the same lines of the same file on their local main, both push. The second push is rejected; the second student must pull, see the conflict, resolve it by editing the file, and complete the merge commit.

4. Pull Request Basics Each student forks a partner's repository, creates a feature branch, makes a small improvement (e.g., adding a feature to the Week 3 organizer script), and opens a pull request. The partner reviews the PR and merges it. Both sides go through the workflow.

5. Mini Project — Team Repository Recovery Challenge (5 Marks) Students work in pairs. The instructor provides a shared GitHub repository containing a project in a slightly broken state — with diverged branches and one or two unresolved conflicts. The pair must:

- Resolve all conflicts collaboratively.
- Approve or comment on the open PR provided.
- Push a clean main branch back to GitHub.
- Submit a shared TEAMWORK.md documenting who did what and how decisions were made.

Both students receive the same mark — explicitly teaching that team outcomes are shared.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

Topics that make team development smoother and more professional.

- 1. SSH Key Authentication** Generate an SSH key with `ssh-keygen`, add it to your GitHub account, and verify with `ssh -T git@github.com`. Understand why SSH is preferred over HTTPS-with-token for daily work.
- 2. Pull Request Review Etiquette** Learn how to leave constructive review comments, request changes, and use suggestion blocks on GitHub. Read a few open-source PRs to see real-world review culture.
- 3. Branch Protection & Code Owners** Explore GitHub repository settings — branch protection rules, requiring reviews before merge, and CODEOWNERS files. Understand how big teams prevent bad code from reaching main.
- 4. Rebasing vs Merging** Understand the difference between `git merge` and `git rebase`. When does each make sense? Read about the 'golden rule of rebasing' — never rebase commits that are already public.
- 5. Working with Forks Long-Term** Add an 'upstream' remote pointing to the original repository, fetch from it, and keep your fork in sync. This is the standard open-source contribution workflow.

WEEK 6 — Python Foundations & Computational Thinking

Conceptual Focus *Transitioning from shell scripting to a general-purpose programming language. Students see Python as a richer, more expressive replacement for Bash when problems outgrow pipelines — while continuing to use the Linux and Git skills they already have.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. Python Environment & Virtual Environments Students install Python 3.11+, create their first virtual environment with `python -m venv .venv`, activate it, install a package (requests) with pip, freeze it to requirements.txt, and add .venv/ to their .gitignore (reinforcing Week 4). They write their first script as a .py file inside their Git-tracked workspace.

2. Variables, Operators & Data Types Students explore Python's built-in types (int, float, str, bool, list, dict). They use the REPL for experimentation and practise string manipulation — slicing, `.split()`, `.strip()`, `.lower()`, f-strings, and `.join()`. They consciously map shell concepts to Python equivalents (`wc -l` → `len(lines)`).

3. Conditions & Loops Students write Python programs using if/elif/else, for and while loops, and break/continue. They learn to read simple tracebacks and use `print()` for debugging.

4. File & Text Processing in Python Students open files with `with open(...)` as f, read line by line, count words, and find substrings. They process a sample text file to produce simple statistics: total words, unique words, and the top-10 most frequent words.

5. Mini Project — Interactive Text Corpus Analyzer (5 Marks) Students build a Python CLI tool `analyze.py` that takes a folder of .txt files (provided by the instructor) and runs as an interactive prompt. The user types commands:

- `stats <filename>` → word count, line count, character count
- `top <n> <filename>` → top n most-frequent words (case-insensitive)
- `search <word>` → list of files containing the word
- `quit` → exit the program
- Tool committed to GitHub with a README.md showing usage examples, running from a clean virtual environment using requirements.txt.

This project plants the seed for the capstone — students are already searching across documents.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

Stretch goals for students who finish the core project early and want to write better Python.

- 1. List Comprehensions & Generator Expressions** Learn the Pythonic way to transform lists: `[w.lower() for w in words if len(w) > 3]`. Understand when to use a generator expression instead for memory efficiency on large files.
- 2. collections.Counter & defaultdict** Replace manual word-counting loops with `Counter(words).most_common(10)`. Explore `defaultdict(list)` for grouping data without checking if keys exist.
- 3. Punctuation Handling with string and re** Use `string.punctuation` and Python's `re` module to strip punctuation before counting words. Compare your top-10 results before and after — they will differ dramatically.
- 4. argparse for Proper CLIs** Replace your interactive prompt with a proper command-line interface using `argparse`. Add `--help` text, subcommands (`stats`, `top`, `search`), and typed arguments.
- 5. Type Hints & Docstrings** Add type hints (`def count_words(text: str) -> dict[str, int]:`) and docstrings to your functions. Run `mypy` to check your types. This is what professional Python code looks like.

WEEK 7 — Web Scraping & Dataset Collection

Conceptual Focus *Moving from local text files to the live web as a data source. Students learn that the documents their future search engine will index can be collected programmatically. The Core Track keeps scraping deliberately simple — fetch a page, extract the entire visible text, save it. Anything more elaborate is in the Exploration Track.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. HTTP Requests with requests Students use the requests library to perform GET requests, inspect status codes, response headers, and content. They contrast requests with command-line tools they already know (curl, wget from Week 1). They learn to handle errors with `response.raise_for_status()` and set a basic User-Agent header.

2. Extracting Plain Text from HTML with BeautifulSoup Students install `beautifulsoup4`, parse a downloaded Wikipedia HTML page, and extract the entire visible text content of the page using `soup.get_text()`. They do NOT split into sections, do NOT extract individual headings or paragraphs separately, and do NOT follow internal links inside the article. The goal is one clean blob of text per article — that is enough for indexing later.

3. Saving Articles to Disk Students save each scraped article as a simple JSON file with a minimal schema: `{title, url, text, fetched_at}`. They reload the saved data and verify nothing was lost. JSON is preferred over CSV because the article body is long and free-form.

4. Scraping a List of URLs Students are given a text file containing a list of 10–15 Wikipedia article URLs. They write a loop that fetches each URL, extracts the plain text, and saves it as a JSON file. They add a `time.sleep(1)` between requests to scrape politely.

5. Mini Project — Wikipedia Dataset Collector (5 Marks) Students build a Python program `collect_wiki.py` that reads a list of Wikipedia URLs from a text file (`urls.txt`) and downloads each one as a clean JSON file. Requirements:

- Reads URLs from `urls.txt` (one per line, provided by the instructor or written by the student).
- For each URL: fetch the page, extract the title and the entire visible text using `soup.get_text()`, and save to `corpus/<slug>.json`.
- JSON schema for each file: `{title, url, text, fetched_at}` — nothing more.
- Polite scraping: `time.sleep(1)` between requests; print which URL is being fetched.

- Handle simple errors: if a URL fails (e.g., 404), print a warning and continue to the next one.
- Script committed to GitHub with a clear README.

The corpus produced this week is the same corpus every subsequent week will operate on. The format is intentionally minimal so that indexing in Week 9 and the capstone is straightforward.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

Everything beyond 'fetch URL, save text' lives here. Students can pick any of these to enrich their dataset.

1. Structured Article Parsing Instead of one big text blob, extract structured fields — the first paragraph as a summary, all section headings, all internal links as a list. Design a richer JSON schema like {title, url, summary, sections: [...], links: [...]}.

2. Following Internal Links (Crawling) Start from a seed article, extract its internal Wikipedia links, and fetch N more articles by following them. Maintain a visited set to avoid re-fetching. Implement BFS or DFS over the link graph. This dramatically grows the corpus but adds complexity around deduplication, redirects, and infinite loops.

3. Retry Logic with Exponential Backoff When a request fails, wait 1 second and retry, then 2 seconds, then 4. Implement this with a small retry loop. Distinguish between transient failures (network blip) and permanent failures (404).

4. Logging with the logging Module Replace print statements with Python's logging module. Configure logging to write to both the console and a collection.log file. Use different log levels (INFO, WARNING, ERROR).

5. Wikipedia API Instead of Scraping Wikipedia provides an official API. Learn to call it with requests and compare the experience — JSON responses are far cleaner than HTML. Discuss when an API should be preferred over scraping.

6. robots.txt & Scraping Ethics Read Wikipedia's robots.txt file. Understand what crawl-delay, Disallow, and User-agent mean. Discuss the ethics of scraping and when permission is needed.

WEEK 8 — Functions, Modularity & Data Structures

Conceptual Focus *Restructuring code from monolithic scripts into reusable, testable units. Students learn that the right data structure (list vs dict vs set) chosen at the right time is often the difference between a slow, fragile program and a fast, robust one — a lesson that pays off enormously in the indexing weeks ahead.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. Function Design, Parameters & Return Values Students refactor their Week 6 corpus analyzer into well-defined functions: `load_corpus(path)`, `word_frequencies(text)`, `search_in_files(word, files)`. They learn to write simple docstrings, use default arguments, and return structured data instead of printing inside functions.

2. Lists, Dictionaries, Sets & Tuples for Text Data Students explore which structure fits which job: List for preserving word order in a document; Dict for mapping word → frequency (the core of indexing); Set for fast membership checks (stopword filtering, deduplication); Tuple for immutable records. They run a small experiment: time how long 'word' in `big_list` takes versus 'word' in `big_set` on a 50,000-word vocabulary. The order-of-magnitude difference is the moment they understand why indexing matters.

3. Multi-File Program Organization Students split a single .py file into a small package: `corpus_tools/` with `__init__.py`, `loader.py`, and `analyzer.py`. They import functions across modules and re-run their Week 6 tool from this new structure with no behaviour change.

4. Processing the Week 7 Corpus Students build a small pipeline that reads the Week 7 Wikipedia JSON corpus and produces a structured output: for each article, a dictionary `{doc_id, title, tokens, unique_token_count}`. The pipeline is composed of small functions, each doing one job.

5. Mini Project — Modular Document Metadata Organizer (5 Marks)

Students build a multi-file Python package `metadata_organizer/` that processes the Week 7 Wikipedia corpus and produces a single `metadata.json` describing the entire collection. Requirements:

- At least three modules (loader, tokenizer, writer), each with focused responsibility.
- Per-document metadata: title, URL, word count, unique word count, top 10 terms.
- Corpus-level metadata: total documents, average length, vocabulary size.

- A main.py entry point that runs the full pipeline.
- Committed to GitHub with a clear module structure and updated README.

This is the first project that genuinely looks like a real software system, not just a script.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

For students who want to write tested, robust, professional Python.

1. Unit Testing with assert and pytest Write a test_organizer.py file with at least five assert-based tests. Then install pytest and convert your asserts into proper pytest functions. Run pytest -v and see the green dots.

2. Length Categorisation & Statistics Add a length_category field (short/medium/long) to each document based on word count thresholds. Compute corpus-level statistics like median length and standard deviation using the statistics module.

3. CLI with argparse Add a proper command-line interface to main.py accepting --corpus and --output arguments. Implement --verbose flag for detailed progress output.

4. Dataclasses for Structured Records Replace dictionaries with @dataclass-decorated classes for Document and CorpusMetadata. Compare readability — type-checked attributes vs string keys.

5. Memory-Efficient Processing with Generators Refactor your loader so that it yields one document at a time using a generator function (yield) instead of returning a giant list. Profile memory usage before and after.

WEEK 9 — Linear vs Inverted Index & Frequency Counting

Conceptual Focus *Introducing the core data structure of every search engine — the inverted index — by first feeling the pain of NOT having one. Students compare linear scanning across documents with index lookup, and understand frequency counting as the simplest form of ranking. Advanced ranking algorithms and fuzzy matching are intentionally moved to the Exploration Track so 2nd semester students can complete the core comfortably.*

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. Linear Search Across Documents Students implement a 'naive' search: for each query word, open every JSON file in the Week 7 corpus, scan its text, and record which files contain the word. They measure how long this takes on their corpus using `time.perf_counter()`. This is the baseline.

2. Building an Inverted Index Students build the central data structure: a Python dictionary mapping each term to the list of documents in which it appears. Format: `{term: [doc_id_1, doc_id_2, ...]}`. They construct it from the Week 7 corpus using the Week 8 tokenizer, then save it to disk as a JSON file so it can be loaded instantly without rebuilding.

3. Comparing Linear vs Inverted Index Students re-run the same queries from activity 1 — but this time using the inverted index. Looking up a word is now a dictionary lookup instead of a scan over all files. They measure and compare runtimes. Seeing the order-of-magnitude speedup makes the value of indexing concrete.

4. Frequency Counting per Document Students extend the index to also store how many times each term appears in each document. New format: `{term: [(doc_id, count), ...]}`. They write a simple function `rank_results(query, index)` that returns documents sorted by total query-term frequency — the simplest possible ranking.

5. Mini Project — Inverted Index Builder & Lookup Tool (5 Marks) Students build a two-part Python tool: (1) `build_index.py` that reads the Week 7 corpus and writes an inverted index to `index.json`; (2) `lookup.py` that loads the index and answers queries from the user. Requirements:

- `build_index.py`: tokenises each document (lowercase, strip punctuation, basic split), builds the inverted index with frequency counts, and saves to `index.json`. (3 marks)

- lookup.py: loads index.json, prompts the user for a query word, and prints the list of documents that contain it, sorted by frequency (most frequent first). (2 marks)
- Demonstrate the speedup: include a small timing comparison in the README showing linear search time vs index lookup time on the same query.
- Committed to GitHub with a clear README explaining how to build and use the index.

By the end of this week, students have a working — though basic — search foundation. The capstone weeks make it real.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

All the algorithmic richness of information retrieval — pushed here so the Core stays achievable. Students who finish Week 9 early can dive into any of these, and the capstone bonus credit explicitly rewards integrating them.

1. Stopword Removal Build a set of common English stopwords ('the', 'of', 'and', etc.) and filter them out during indexing. Compare index size and query result quality before and after. Discuss the trade-off: stopwords can be important for some queries ('to be or not to be').

2. TF-IDF Ranking Implement TF-IDF from scratch: Term Frequency $tf(t,d) = \text{count}(t \text{ in } d) / \text{len}(d)$; Inverse Document Frequency $idf(t) = \log(N / df(t))$; Score = $tf \times idf$. Run a query like 'machine learning' and compare TF-IDF ranking against simple frequency ranking. TF-IDF surfaces more relevant documents because it discounts common terms.

3. Storing Positions for Phrase Queries Extend the index from {term: [(doc_id, count)]} to {term: [(doc_id, count, [positions])]} where positions are word offsets within the document. This unlocks phrase queries ('New York Times' as an exact phrase) by checking that consecutive query terms have consecutive positions in the same document.

4. Boolean Query Support Implement AND, OR, NOT between query terms using set operations on posting lists. 'python AND linux' is the intersection of the two posting lists; 'python OR linux' is the union; 'python NOT windows' is set difference.

5. Fuzzy Matching — Edit Distance & difflib Implement Levenshtein edit distance (recursively first to understand it, then iteratively with dynamic programming). Use Python's built-in difflib.get_close_matches for a quick 'did you mean?' feature: when a query word isn't in the vocabulary, suggest the three closest terms.

6. Binary Search on Sorted Vocabulary Sort the vocabulary alphabetically and implement binary search on it. Measure runtime on a 10,000-term vocabulary using `time.perf_counter()` and compare to linear search and dictionary lookup. Understand when sorting is worth it and when it isn't.

WEEK 10 — Indexing & Search Architecture (Capstone Transition)

Conceptual Focus *From this week onward, students transition into capstone development with full freedom on implementation. The goal of this week is to take the Week 9 inverted index and integrate it into a complete query engine that becomes the foundation of the capstone.*

This week is not separately graded as a 5-mark mini project. The work done here directly feeds the capstone evaluation in Weeks 11–12.

CORE TRACK — Mandatory (5 marks)

Laboratory Activities

1. Text Cleaning & Tokenization Improvements Students improve their tokenizer: consistent lowercasing, stripping punctuation more carefully, and handling edge cases (numbers, hyphenated words). They rebuild their index on a clean corpus.

2. Posting Lists & Document Mapping Students formalize the index into a documented posting-list structure and a separate document map ({doc_id: {title, url, length}}). They serialise both to disk and confirm the index loads instantly on subsequent runs.

3. Query Retrieval Workflow Integration Students combine the index, document map, and a simple ranking function into a single ranked_search(query) function. The function takes a query string, tokenises it, looks up each term in the index, ranks the results by total frequency, and returns the top 5 documents.

4. Capstone Planning Each student (or team) submits a one-page architecture sketch of their capstone — module diagram, data flow from query to ranked result, and the list of optional features they commit to building from the Exploration Track.

5. Capstone Foundation Project — Ranked Query Engine A baseline working query engine that loads the index from disk, accepts a query, and returns the top-5 ranked documents. This becomes the starting point for the capstone in Weeks 11–12.

EXPLORATION TRACK — Optional / Ungraded

Exploration Activities (Optional, Ungraded)

Students are encouraged to pick features from the Week 9 Exploration list and integrate them into their capstone for bonus credit during evaluation.

- Add stopwords filtering to the indexer.
- Replace simple frequency ranking with TF-IDF.
- Add boolean query operators (AND / OR / NOT).
- Add fuzzy 'did you mean?' suggestions for typos.
- Add snippet highlighting (show the matched word in context).
- Build a small Streamlit or Flask web UI on top of the CLI.

WEEKS 11 & 12 — Final Capstone Development & Demonstration

Conceptual Focus *Full system integration. Students bring together every skill from the course — Linux, scripting, Git collaboration, Python modularity, scraping, data structures, indexing, and ranking — into a single working product.*

Laboratory Activities

1. Final Capstone — Intelligent Wikipedia Search Engine

The complete integrated system must include the following Core components:

- **Dataset** — A corpus of at least 50 Wikipedia articles collected with the Week 7 scraper.
- **Index** — A persistent inverted index built once and loaded on startup.
- **Ranking** — Frequency-based ranked retrieval showing top-N results. (TF-IDF is bonus, not required.)
- **Query handling** — Support for multi-word queries and case-insensitive matching.
- **Interface** — A polished CLI (mandatory).
- **Engineering quality** — Modular Python package, virtual environment, requirements.txt, meaningful Git history on GitHub, README.md with setup and usage instructions.

2. Bonus Features (From the Exploration Track) Earn bonus credit during demonstration by integrating any of these:

- Fuzzy 'did you mean?' suggestions for misspelled queries.
- Query autocomplete / suggestions.
- Boolean operators (AND / OR / NOT).
- TF-IDF ranking instead of simple frequency.
- Phrase queries using position-aware index.
- Streamlit or Flask web UI.
- Architecture diagram in the README.

3. Final Demonstration In the last lab session, each student or team demonstrates their system live:

1. Run a fresh setup from a clean clone of their GitHub repo.
2. Execute three queries chosen by the evaluator.
3. Demonstrate any bonus features they have built.
4. Explain one part of their code chosen at random by the evaluator.

5. Answer questions about one design decision they made.